

Medovik: Simplifying Porting to Trusted Execution Environments

Lucian MOCAN

Supervised by Peter DINDA

Host Institution: *Prescience Lab, Northwestern University*

 **Northwestern** | McCormick School of
ENGINEERING



August 25th, 2026

Outline



- 01** **Internship context and my role**
- 02** **Why porting to Trusted Execution Environments (TEEs) is difficult**
- 03** **The Medovik recipe**
- 04** **Baking the prototype**
- 05** **Tasting the results**
- 06** **Conclusion: Outcomes and lessons learned**

01 Internship context and my role



Northwestern University

McCormick Engineering

Computer Science

Prescience Lab

(17 current members)

Privacy Backplane

Erase + Medovik

01 Internship context and my role



Collaboration & Ownership

My contribution

- hardware integration + system design
- prototype implementation + evaluation
- paper drafting

Collaborated closely with

- Prof. Peter Dinda: weekly meetings & direction
- Michael Polinski: testbed + co-authorship
- Friedrich Doku: feedback + discussions

Northwestern University

McCormick Engineering

Computer Science

Prescience Lab

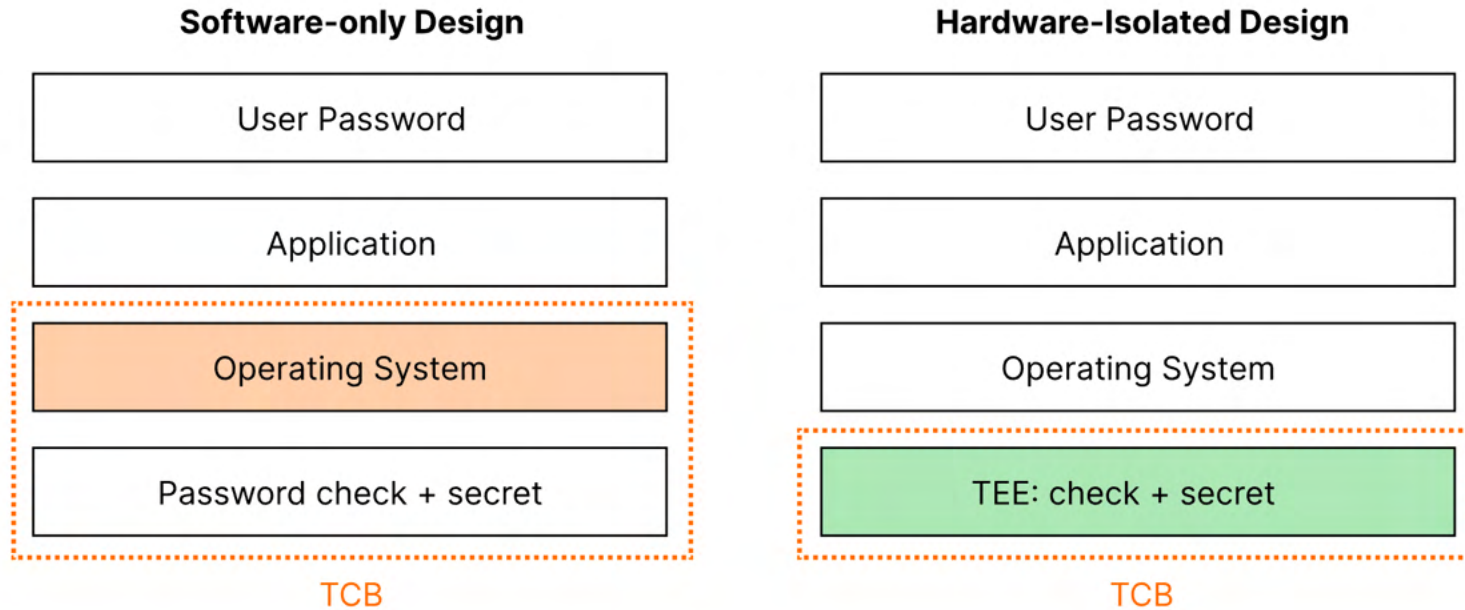
(17 current members)

Privacy Backplane

Erase + Medovik

02

Moving trust into hardware

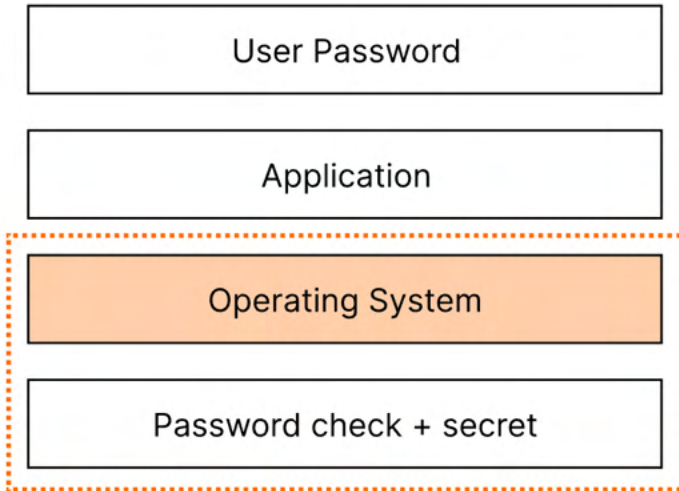


02

Moving trust into hardware

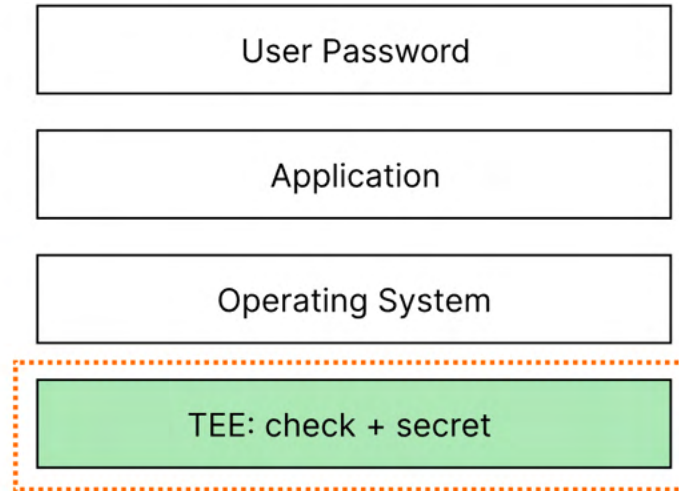


Software-only Design



TCB

Hardware-Isolated Design



TCB

A TEE protects code and data from the rest of the system, including the ordinary OS.

Attestation

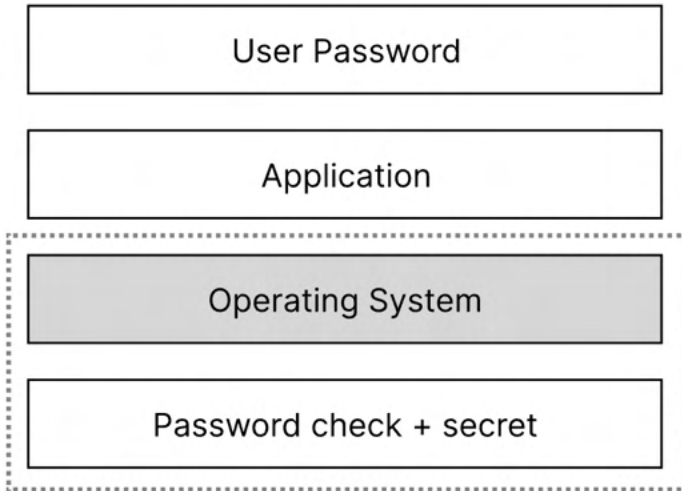
proves that the expected code was loaded inside a real TEE.

02

Moving trust into hardware

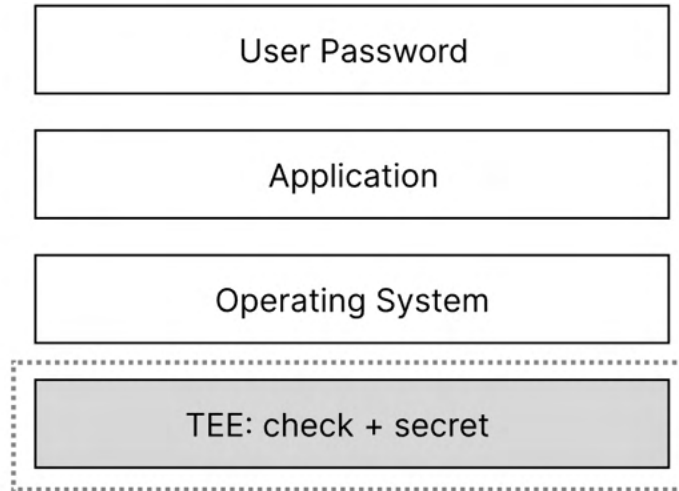


Software-only Design



TCB

Hardware-Isolated Design



TCB

A TEE protects code and data from the rest of the system, including the ordinary OS.

Attestation

proves that the expected code was loaded inside a real TEE.

x86 : SGX, **TDX**, **SEV-SNP** (for Virtual Machines support)

ARM: TrustZone (proposed Realms Management Extension (RME) in ARMv9 [2] but **no hardware since 2021**)

02 Why porting to TEEs is



Showcase: ERASE [3,4]



Figure 1. ERASE processes video at the edge and redacts people who have not opted into image capture. [3]

02 Why porting to TEEs is



Showcase: ERASE [3,4]



Figure 1. ERASE processes video at the edge and redacts people who have not opted into image capture. [3]

Existing ERASE stack:

x86_64 Ubuntu container
Python + NumPy + SciPy
OpenCV



Target

ROCKPro64 ARM64
TrustZone Secure World
TEE OS - dependent execution model



runtime
libraries
shared objects
drivers

03 The Medovik recipe

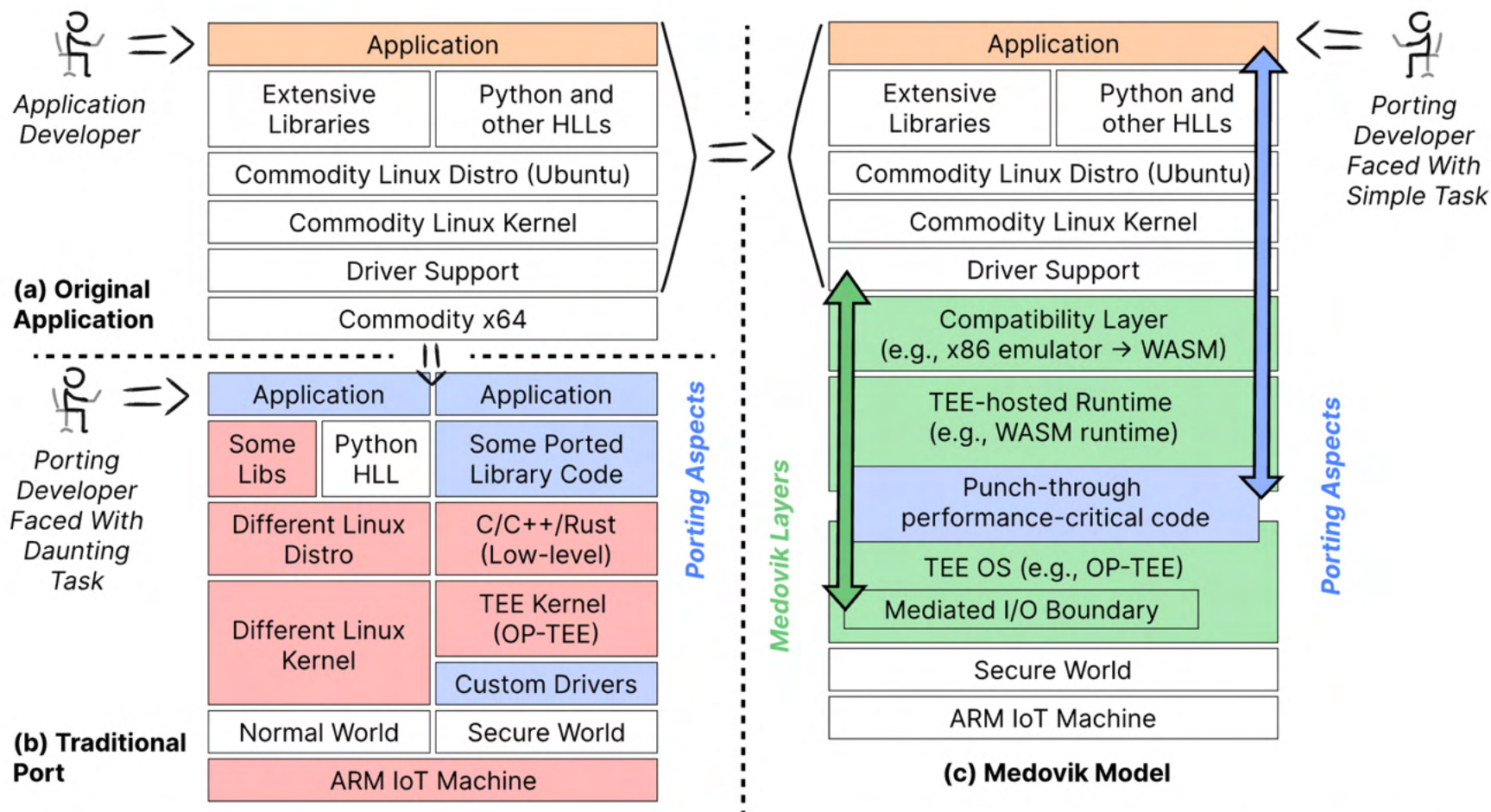
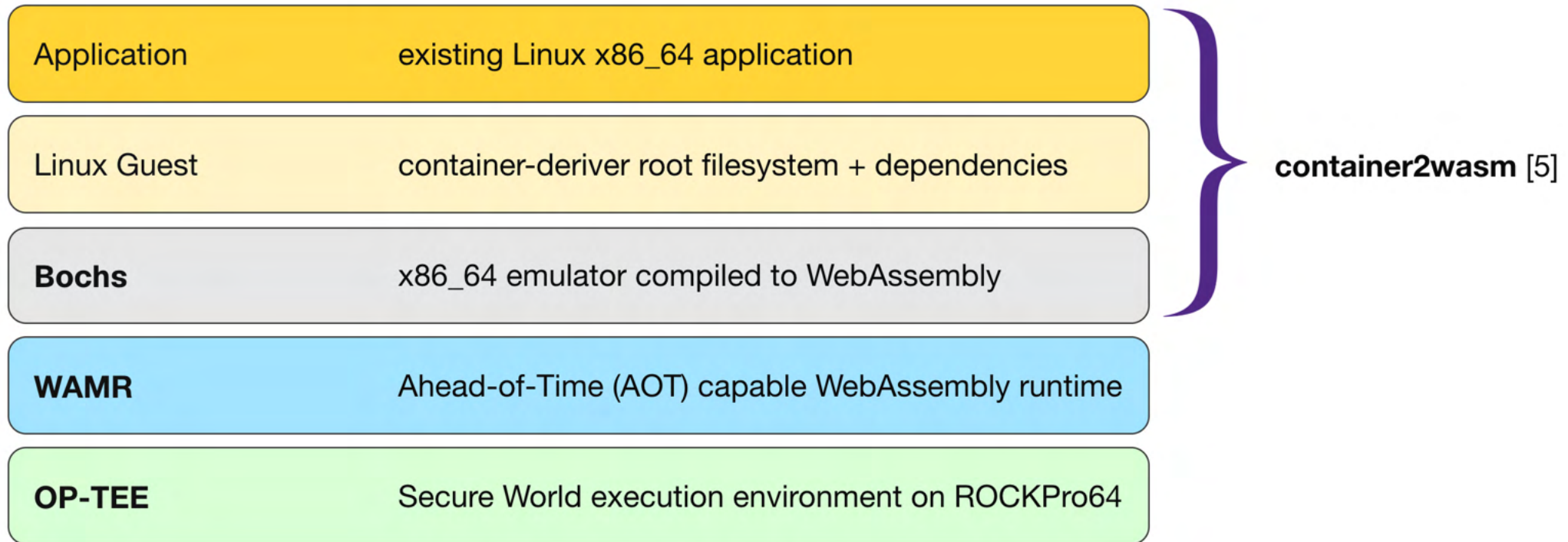


Figure 2. **Medovik simplifies porting to TEEs.** (a) Applications are built for commodity x86_64/Linux stacks. (b) Traditional TEE porting forces developers to split or replace application, runtime, OS, and I/O layers across Normal and Secure Worlds. (c) **Medovik preserves the original stack inside TEE-hosted compatibility layers, then uses profiling-guided punch-through for selected hot paths.**

04 Baking the prototype



04 Baking the prototype



Application	existing Linux x86_64 application
Linux Guest	container-driver root filesystem + dependencies
Bochs	x86_64 emulator compiled to WebAssembly
WAMR	Ahead-of-Time (AOT) capable WebAssembly runtime
OP-TEE	Secure World execution environment on ROCKPro64

container2wasm [5]



05

Tasting the results

Compatibility works (large slowdown)



100%

workloads completed without application porting

125.55x

geometric-mean compatibility-first slowdown

~24x

ERASE with QEMU directly on ARM

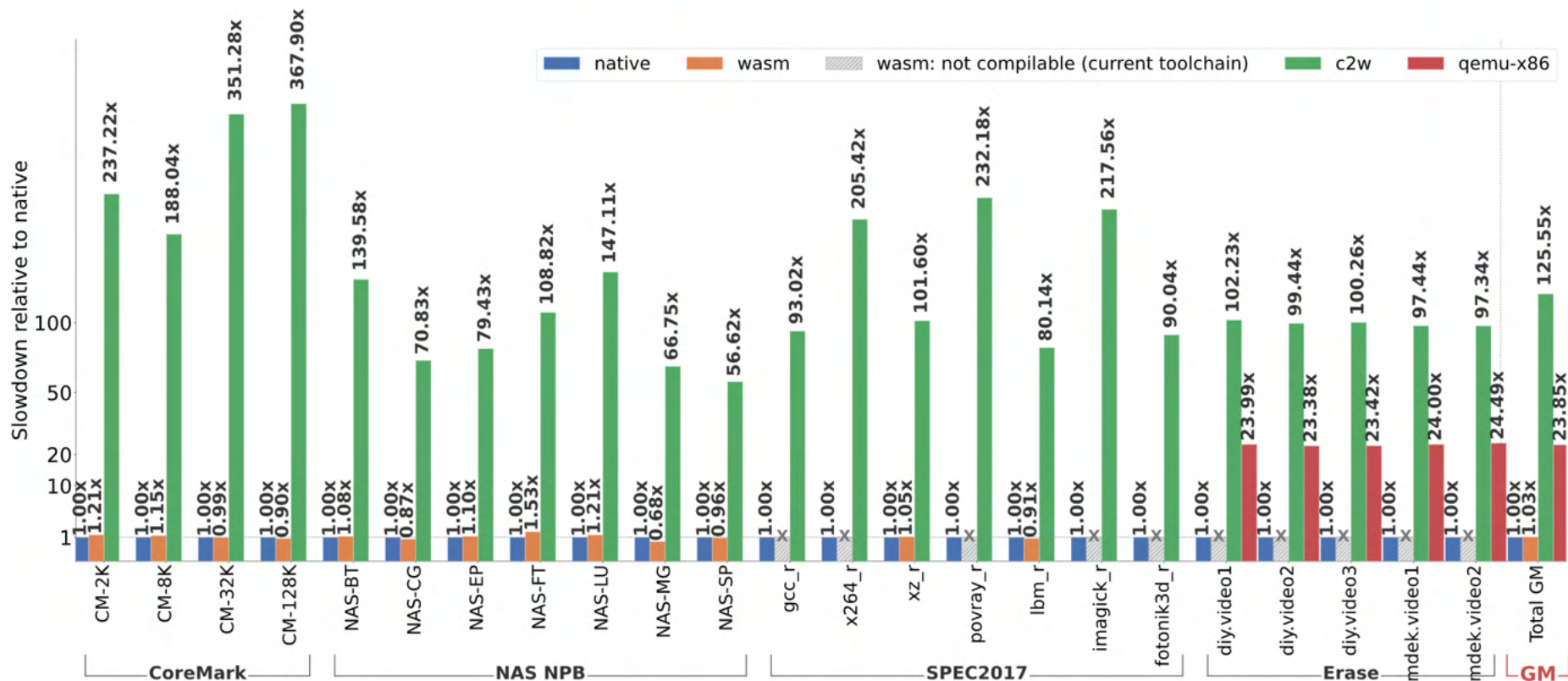


Figure 3. Compatibility-first execution requires no porting effort but incurs substantial slowdown. Direct WebAssembly closely matches native performance where supported.

Hot paths make punch-through plausible



38.3%

geometric mean: hottest
function

69.9%

geometric mean: top three
functions

84.2%

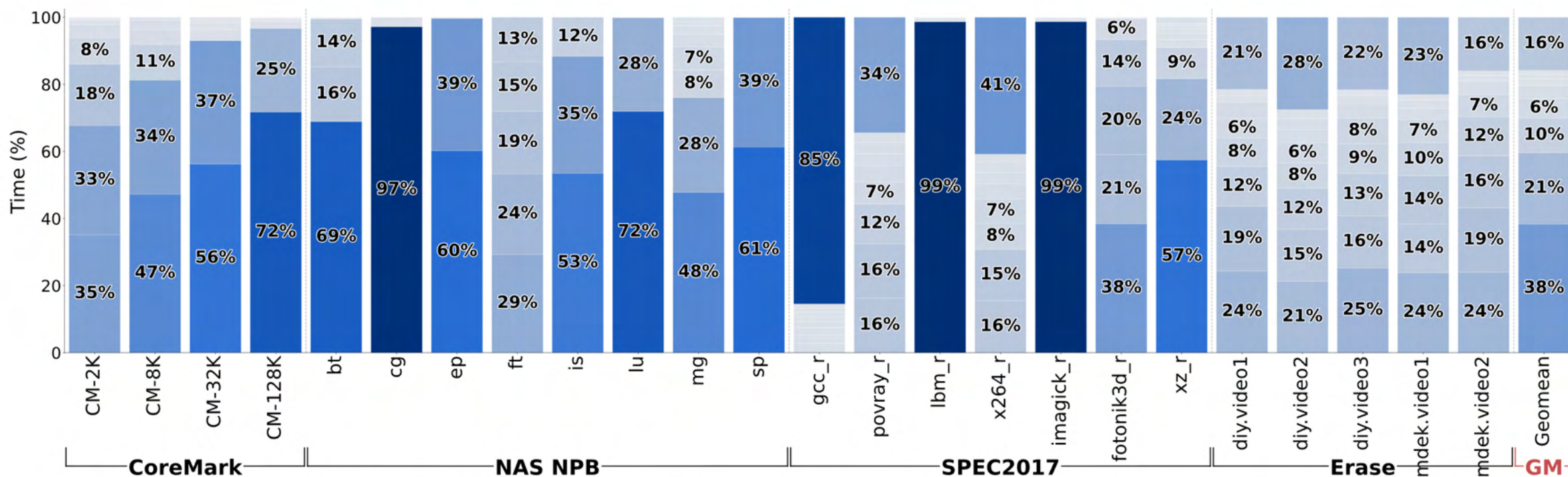
geometric mean: top eight
functions

Figure 4. Profiled execution time by function across workloads. Blue segments show individual functions or groups, with darker shades indicating larger shares; the light segment combines all remaining functions. Execution time is often concentrated in a small number of hot functions.

05 Punch-through redirects selected guest calls



Guest call

Shim

cpuid hypercall

C++ handler

```
1 import wasm_clock
2 ...
3 def _now_sec():
4     return float(_WASI_TIMER())
5 ...
6 start_t = _now_sec()
7 ...
8 print("Total Runtime: ", _now_sec() - start_t)
```

05 Punch-through redirects selected guest calls



Guest call

Shim

cpuid hypercall

C++ handler

```
1 static PyObject* wasi_timer(PyObject* self, PyObject* args) {
2     uint32_t eax = 0xDEADBEB0;
3     uint32_t ebx = 0;
4     uint32_t ecx = 0;
5     uint32_t edx;
6
7     __asm__ volatile (
8         "cpuid"
9         : "+a" (eax), "+b" (ebx), "+c" (ecx), "=d" (edx)
10        :
11        : "memory"
12    );
```

05 Punch-through redirects selected guest calls



Guest call

Shim

cpuid hypercall

C++ handler

```
1 #include <ctime>
2 void BX_CPP_AttrRegparmN(1) BX_CPU_C::CUID(bxInstruction_c *i){
3     if (EAX == 0xDEADBEB0) {
4         ...
5         clock_gettime(CLOCK_MONOTONIC, &ts); // C/C++ handler
6         ...
7         RBX = GET32L(sec_bits);
8         RCX = GET32H(sec_bits);
9         RDX = 0;
10        BX_NEXT_INSTR(i);
11        return;
12    }
```

05 Punch-through redirects selected guest calls



Guest call

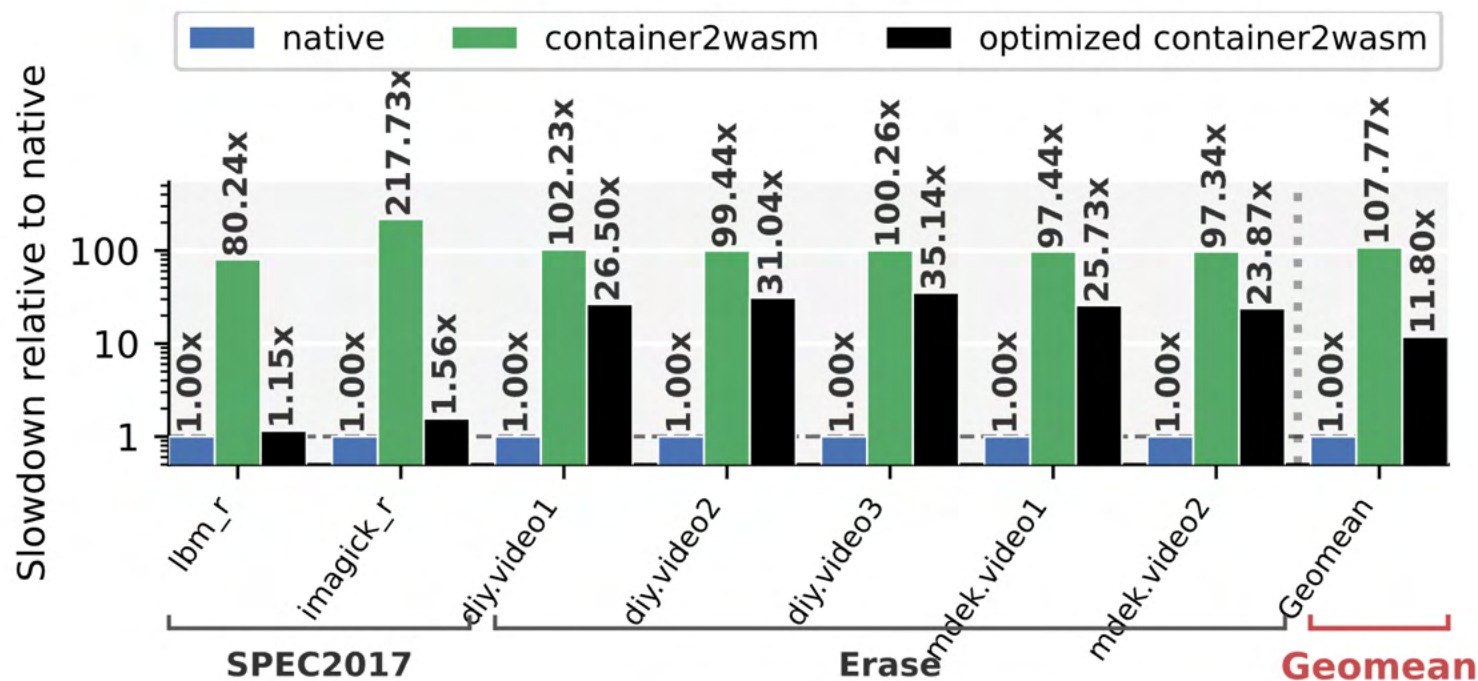
Shim

cpuid hypercall

C++ handler

```
28  /* Get current value of CLOCK and store it in TP. */
29  int
30  __clock_gettime64 (clockid_t clock_id, struct __timespec64 *tp)
31  {
32  ...
33  # ifdef HAVE_CLOCK_GETTIME64_VSYSCALL
34  return INLINE_VSYSCALL (clock_gettime64, 2, clock_id, tp);
35  # else
36  return INLINE_SYSCALL_CALL (clock_gettime64, clock_id, tp);
37  # endif
38  ...
```

05 Punch-through substantially reduces slowdown



lbm_r

80.24x



1.15X

imagick_r

217.73x



1.56X

ERASE

~99x



~28X

Figure 5. Punch-through of selected hot functions substantially reduces slowdown.

06 A working model, but...



What the results support

- unmodified x86_64/Linux apps can run inside ARM TrustZone with **no effort**
- profiling can narrow manual porting to selected hot paths
- targeted punch-through can **recover substantial performance**

What needs improvement

- Bochs is **super** *slow*
- WAMR + emulator enlarge the trusted computing base
- Secure camera / USB passthrough is still missing
- Benefits depend on hot-path concentration

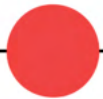
Research outcomes and what comes next



Prototype

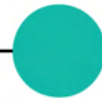


Poster



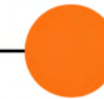
May 11th at
GCASR 2026

Paper



PLOS 2026 (co-hosted
with SOSP) rejection

Next



improved baseline + secure
device passthrough

06 What I've learned



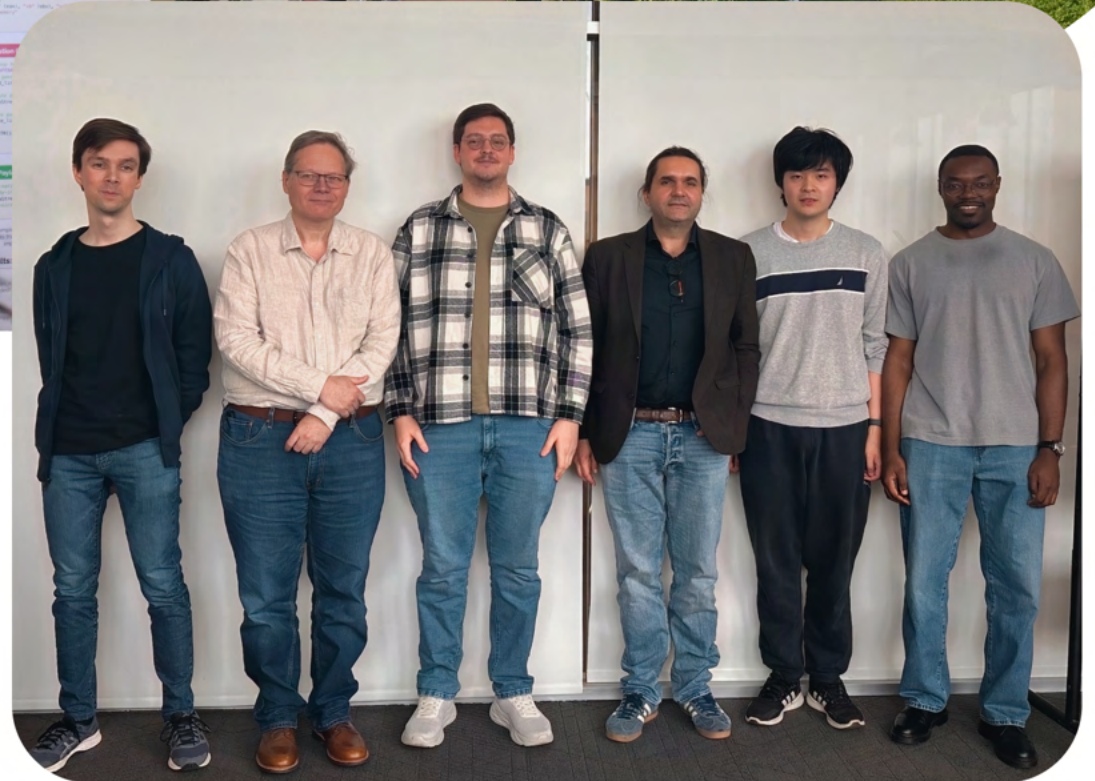
- TEEs, WebAssembly runtimes, profiling, performance evaluation, emulators
- Teamwork, discussions and technical feedback, research direction
- Writing a paper, researching literature, choosing a research idea/topic
- Respecting a deadline, identifying a potential publish venue
- Present your work in front of other people (FIGURE and BTAA meeting on June 26th)
- Can agentic LLMs really port entire codebases?
- Many other things

Thank you for your attention!



Special thanks to:

- Prof. Peter Dinda
- Michael Polinski (PhD candidate)
- Friedrich Doky (PhD candidate)
- David Krasowska (PhD candidate)
- Peizhi Liu (PhD candidate)
- Henron Ruan (undergraduate student)



References



- [1] "Trusted computing base," Wikipedia, The Free Encyclopedia. Available: https://en.wikipedia.org/wiki/Trusted_computing_base.
- [2] Learn the Architecture: Realm Management Extension Guide. DEN0126. Arm Limited. URL: <https://developer.arm.com/documentation/den0126/0102/Overview>.
- [3] Haotian Qiao et al. "Efficient Video Redaction at the Edge: Human Motion Tracking for Privacy Protection". In: ACM Transactions on Embedded Computer Systems 24.5s (Sept. 2025).
- [4] Haotian Qiao et al. "Efficient Video Redaction at the Edge: Human Motion Tracking for Privacy Protection". In: Proceedings of the International Conference on Hardware/Software Codesign and System Synthesis (CODES-ISSS 2025). Oct. 2025.
- [5] Cloud Native Computing Foundation. container2wasm. URL: <https://www.cncf.io/projects/container2wasm/>.
- [6] Jarred Sumner. Rewriting Bun in Rust. July 2026. URL: <https://bun.com/blog/bun-in-rust>.
- [7] Joseph Savona. [compiler] Port React Compiler to Rust. Merged pull request #36173 in react/react on June 9, 2026. Mar. 2026. URL: <https://github.com/react/react/pull/36173>

06 A quick dive into the world of Large Language Models



Main challenge : **porting**

Why codebases move

- long-lived and legacy codebases
- incompatible architecture and envs
- constrained TEE toolchains and APIs
- migrations to memory-safe languages
- shrinking pool of specialist expertise

Current options:

- rewrite from scratch
- port the code manually
- partition the system or rebuild missing libraries
- assemble a team of domain experts

06 Can agentic LLMs really port entire codebases?



Bun: Zig -> Rust [6]

- **successful** rewrite (after more than 3 months, it was finally released in Bun 1.4 on August 20)

~535k

lines of Zig
rewritten

11 days

of LLM-driven
work

~\$165k

API-price
estimate

**How much evidence is needed
for an LLM-translated program
to be considered correct?**

React Compiler: TypeScript -> Rust [7]

... more to come